DETECTive: ML-driven Automatic Test Pattern Prediction

Faults in Digital Circuits — A GNN+LSTM Approach to ATPG

Prasham Shah | Dhruvi Jobanputra

April 2026

BTech Mini Project
VLSI Testing and Validation
Dhirubhai Ambani University

Block 1 — Traditional ATPG & Motivation

Why ATPG?

- Every fabricated chip must be tested for **manufacturing defects** before shipment
- Defects are modelled as **logical faults** on internal nets; test patterns must expose them at primary outputs
- **Automatic Test Pattern Generation (ATPG)** solves this via deterministic search — D-Algorithm, PODEM, FAN
- **Bottleneck:** the search space scales as $2^{N_{PI}}$; for industrial designs ($N_{PI} > 10^4$) this is intractable without strong heuristics

Stuck-at Faults — The Standard Fault Model

Stuck-at Fault Model

- A net is permanently tied to a logic value, regardless of the driving gate
- **SA0**: net stuck at logic 0
- **SA1**: net stuck at logic 1
- Captures opens and shorts to power / ground

Two requirements for detection:

1. Activate — drive the faulty net to the *opposite* value in the fault-free machine
2. Propagate — sensitise a path from fault site to a primary output

Fault on output net Z:
activate by A=1, B=1
(good Z=1, faulty Z=0)

The Exponential Problem

Why simple searching fails

- For N_{PI} inputs, there are $2^{N_{PI}}$ possible test patterns
- ISCAS-85 *c1908* has 33 PIs $\Rightarrow$ Total search space of $2^{33} \approx 8.6 \times 10^9$ patterns
- Modern chips have $N_{PI} > 10^4$ (more patterns than atoms in the universe)
- Testing every fault across thousands of gates becomes impossible

Traditional ATPG Solutions

- Use **smart search trees** to find patterns instead of guessing randomly
- Evolution of tools: D-Algorithm $\rightarrow$ PODEM $\rightarrow$ FAN (each faster than the last)
- **The Limit:** They are all "brute-force" at heart; they still slow down exponentially on complex chips

D-Algorithm — Roth, 1966

Roth's D-Algebra represents **good** and **faulty** machine values *simultaneously* on every net.

The five values

0	good = 0, faulty = 0
1	good = 1, faulty = 1
D	good = 1, faulty = 0
$\overline{D}$	good = 0, faulty = 1
X	unknown / don't care

Derivation flow for any PDCF

1. **Goal** — determine the discrepancy required at the gate output for the given fault
2. **Singular Cover row** — pick the gate-truth-table row whose output matches the goal
3. **Good vs. Faulty Evaluation** — evaluate the same input under fault-free *and* faulty machines
4. **Final Vector** — encode the (good, faulty) pair as a single D-algebra symbol

D-Frontier: The set of gates whose inputs carry a fault (D or $\overline{D}$) but whose outputs are currently unassigned (X). It defines the potential paths for fault propagation.

PDCF Derivation — 2-Input AND Gate, Output SA0

Primitive D-Cube of Failure (PDCF): the minimal input assignment that produces D at the gate output for a given fault.

Step-by-step derivation:

1. **Goal.** Force the *good machine* output to 1 so that SA0 produces a discrepancy:

$$Z_{\text{good}} = 1, \quad Z_{\text{faulty}} = 0 \implies Z = D$$

2. **Singular Cover row.** Pick the AND truth-table row giving 1: $(A, B, Z) = (1, 1, 1)$.
3. **Good vs. Faulty Evaluation.** $Z_{\text{good}} = A \wedge B = 1$, while $Z_{\text{faulty}} = 0$ by SA0.
4. **Final Vector.** The pair $(1, 0)$ is encoded as the symbol D.

PDCF for AND-gate output SA0

$$\boxed{A = 1, \quad B = 1 \implies Z = D}$$

Working of D-Algorithm

Four phases, looped until detection or contradiction:

1. **Fault Activation** — apply the PDCF at the fault site, placing D or $\overline{D}$
2. **D-Drive (Forward)** — pick a gate on the D-frontier, apply its propagation D-cube to push the fault effect toward a primary output
3. **Line Justification (Backward)** — recursively justify any required 0/1 assignments by setting upstream lines
4. **Backtrack on Conflict** — if any line is forced to two contradictory values, undo the last decision and try the alternative

Why it explodes

Nested justifications on internal lines, plus value conflicts at reconvergence stems, trigger exponential backtracking.

Limitations of D-Algorithm

The first complete ATPG algorithm

- Operates on *any* signal line (internal or PI)
- Two phases: D-drive (push *D* toward outputs) and consistency check (justify required values back to PIs)
- Provably complete — finds a test if one exists
- Foundation for all later structural ATPG

The cost

- Decisions on *internal* nets often conflict with PI assignments
- Backtracking grows **exponentially** with reconvergent fanout
- Justification often dominates runtime

Worst-case complexity

Reconvergent circuits push runtime exponential — motivating the move to PI-only decisions (PODEM) and structural headlines (FAN).

Classical ATPG: PODEM (Goel, 1981)

The Breakthrough: Instead of guessing internal wire values, restrict every decision strictly to the *Primary Inputs*. Let deterministic *forward simulation* handle the rest.

Why this changes everything

- The search space collapses from 2^k (all internal gates) down to 2^n (primary inputs).
- Since $n \ll k$, this physically prevents the exponential explosions seen in the D-Algorithm.
- Forward simulation only: No more guessing internal wire values that lead to logical conflicts.

Two key heuristics

- **Objective:** Identify a target (e.g., "activate the fault" or "push the D-Frontier forward").
- **Backtrace:** Walk backward from that target, through the logic gates, until a free Primary Input is found.

Working of PODEM — The X-Path Check

The Search Loop

1. Identify a target objective.
2. Backtrace to find an unassigned Primary Input.
3. Assign a 0 or 1 to that input and forward-simulate the circuit.
4. Check: Did the fault signal move closer to the output?
5. If not, backtrack and flip the input value.

The X-Path Check (The Safety Net)

- Before making deep decisions, PODEM runs a quick Breadth-First Search (BFS) forward from the fault signal.
- It checks if a physical path of unassigned gates (X) still exists to the output.
- If the signal is trapped, it stops and backtracks immediately.

Worked PODEM Example — 4-Gate Circuit

Circuit Logic: $y = \mathrm{NOT}(\mathrm{AND}(\mathrm{AND}(a, b), \mathrm{OR}(c, d)))$
Target Fault: Inner AND-gate output stuck-at-0.

Step	Objective	PI Assignment & State
1	Activate fault: $\{n_1, 1\}$	$[a = 1]$
2	Continue activation: $\{n_1, 1\}$	$[a = 1, b = 1] \Rightarrow$ Fault active (D)
3	Propagate via AND (require OR$= 1$)	$[a = 1, b = 1, c = 1]$
4	X-Path Check	Confirms PO (y) remains reachable
5	Final Simulation	Output $y = \bar{D}$ (Fault Detected)

Final Test Vector

$(a = 1, b = 1, c = 1, d = X)$ — The 1-bit don't-care (X) is preserved, demonstrating algorithm efficiency.

Limitations of PODEM

The Wall: While PODEM drastically reduces the search space, it still relies on blind primary input assignments. This creates bottlenecks in modern, complex designs.

Where PODEM struggles

- **Reconvergent Fanout:** When paths split and rejoin, assigning an input to help one branch often inadvertently sabotages the other.
- **Inefficient Backtracking:** PODEM might discover a conflict deep inside the circuit, but it is forced to walk all the way back to the primary inputs to fix it.

The Next Step

To solve these inefficiencies, the industry transitioned to **FAN (Fan-out Oriented Test Generation)**, which stops backtracing at internal "headlines" to catch conflicts much earlier.

FAN — Fanout-Oriented Algorithm

Building on PODEM, FAN further reduces backtracking by exploiting circuit topology.

- **FAN Algorithm** (Fujiwara & Shimono, 1983)
- **FAN = Fanout-Oriented Algorithm**
  → Designed to reduce backtracking in ATPG
- **Improves over D-Algorithm**
  → Uses circuit structure to guide search
- **Classifies circuit wires into:**
  - Fanout points → signal splits into multiple paths
  - Bound lines → affected by reconvergence
  - Free lines → no reconvergence (easy to handle)
  - Headlines → boundary where backtracking is limited

Working of FAN

- **Fault Assignment & Implication**
  → Assign fault and propagate values forward / backward
- **Unique Sensitisation**
  → If only one path exists, assign non-controlling values directly
- **Multiple Backtrace** (Smart Decision)
  → Uses voting + SCOAP to choose the best input
- **X-Path Check**
  → Avoids paths that cannot reach an output
- **Free Line Justification**
  → Solve simple regions without backtracking

The classical-ATPG ceiling

Even with FAN's heuristics, runtime grows super-linearly with circuit size. Industrial tools are FAN descendants and remain the bottleneck in modern test flows.

Motivation: Why we need ATPP?

The Old Way: Classical Search

- Signals split and rejoin due to reconvergence. A choice made at one gate often breaks a requirement at another.
- The algorithm is forced to backtrack, undoing its work and guessing again thousands of times.
- Computation time explodes exponentially as chips get more complex.

The New Way: ATPP Prediction

- Instead of guessing and checking, we analyze the circuit structure as a whole.
- A neural network outputs the entire test pattern in a single forward pass.
- Eliminating backtracking keeps the runtime flat and predictable, even for deep logic.

From Search to Intelligence

Traditional ATPG burns time on blind search. DETECTive uses machine learning to understand circuit intent, replacing millions of guesses with one smart prediction.

Click here to view ATPG vs. ATPP Search Complexity Animation

Block 2 — ATPP Architecture

ATPP — Replacing Search with Prediction

Automatic Test Pattern Prediction (ATPP) is a learned mapping that takes a faulted netlist and outputs a fault-exposing input pattern in one pass.

- There is no backtracking and no decision tree. A feed-forward neural network produces the answer directly.
- **The Pipeline:** GNN encoder (graph $\rightarrow$ node embeddings) + Two LSTMs (variable paths $\rightarrow$ fixed vectors) + MLP (cone $\rightarrow$ PI prediction).
- The model is trained on synthetic 4-PI circuits using brute-force enumeration ($2^4 = 16$ combinations) to provide exact ground-truth patterns.

The Core Shift

Classical ATPG searches a decision tree and backtracks on conflicts. ATPP predicts the pattern directly from the physical structure of the circuit.

Circuit as a Graph

Step 1: Convert the gate-level netlist into a format the neural network can see.

Graph $G = (V, E)$

- Each gate and primary input becomes a node.
- Each wire becomes a directed edge pointing from the driver to the driven gate.
- Edges strictly follow the combinational signal flow.

Why build a graph?

- It captures the true circuit topology without flattening it into a rigid matrix.
- It allows the neural network to pass information along the exact same paths the electrical signals travel.

Data Shape Tracker

We now have a mathematical graph containing N nodes.

Node Feature Tensor — Giving the Graph Meaning

Step 2: Attach a mathematical feature vector to every single node.

The 11 dimensions per node:

- **9 dimensions:** A one-hot encoding of the gate type (INPUT, AND, NAND, OR, NOR, XOR, XNOR, NOT, BUF).
- **1 dimension:** $\ln(1 + \mathrm{fanout})$. Log-scaling prevents high-fanout nets from blowing up our gradients during training.
- **1 dimension:** The `is_faulty` flag. It is set to 1.0 specifically on the fault site and 0 everywhere else. This is the anchor that tells the network exactly where to focus.

Data Shape Tracker

The graph is now represented by an input tensor $\mathbf{X} \in \mathbb{R}^{N \times 11}$. Every single one of the N nodes carries 11 distinct features.

GNN Encoder — Graph Attention and Convolution

Step 3: Process the graph so the gates understand their local neighborhoods.

1× GAT layer (Graph Attention)

- Learns importance weights for each path.
- Fan-out is asymmetric; not all branches are useful. GAT pays selective attention to paths that actually propagate the fault.

2× GCN layers (Graph Convolution)

- Smooths and diffuses the feature data across the graph.
- Two layers extend the receptive field, ensuring each node is aware of its 2-hop neighborhood.

Data Shape Tracker

The GNN processes the raw $\mathbf{X} \in \mathbb{R}^{N \times 11}$ input and generates rich spatial embeddings. The new tensor shape is $\mathbf{H} \in \mathbb{R}^{N \times 32}$.

Path Extraction — Bridging Graph to Sequence

Step 4: Extract physical routes from the spatial graph to prepare data for LSTM processing.

Tracing the Routes (Hybrid BFS+DFS)

- **Activation Paths:** BFS finds the shortest PI-to-fault path; capped DFS gathers additional routes.
- **Propagation Paths:** BFS identifies the fastest path to any PO; DFS "tops up" the remaining budget.
- Result: A sequence of Gate IDs representing continuous electrical paths.

Budgeting and Constraints

- **Path Cap:** Restricted to **10 activation** and **10 propagation** paths (First-found wins).
- **Search Guard:** DFS limited to a 500-node horizon to prevent exponential blow-up.

Data Shape Tracker

Static Graph ($\mathbf{H} \in \mathbb{R}^{N \times 32}$) $\longrightarrow$ Sequential Paths ($\mathbf{S} \in \mathbb{R}^{1 \times L \times 32}$).

Two LSTMs — Handling Variable Length Paths

Step 5: Extract paths and compress them. A path can be 3 gates long or 30 gates long. Neural networks require fixed sizes, so the LSTM compresses this variable length (L) into a static vector.

Activator LSTM

- Consumes activation paths (PI $\rightarrow$ fault site).
- We must append the specific fault-type bit (SA0 / SA1) to the final hidden state so the network knows what value to force.

Propagator LSTM

- Consumes propagation paths (fault site $\rightarrow$ PO).
- No fault-type is needed here, because propagating an error is fault-agnostic once excited.

Data Shape Tracker

The LSTM takes variable-length sequences ($\mathbb{R}^{1\times L\times 32}$). The Activator outputs a fixed $\mathbb{R}^{33}$ vector. The Propagator outputs a fixed $\mathbb{R}^{32}$ vector.

Cone Embedding — Fusing the Physical Context

Step 6: A fault might have hundreds of possible paths. We cap extraction at 10 activation paths and 10 propagation paths to create a single, unified descriptor.

- **Activation slot:** 10 paths $\times$ 33 dimensions $= 330$
- **Propagation slot:** 10 paths $\times$ 33 dimensions $= 330$
- If a fault has fewer than 10 paths, we zero-pad the missing slots. This guarantees our network always receives the exact same input size.

Data Shape Tracker

We concatenate these slots to form the Cone Matrix: $\mathbf{C} \in \mathbb{R}^{660}$.

The Final MLP Predictor

$$\mathbf{C} \in \mathbb{R}^{660} \longrightarrow \mathrm{MLP} \longrightarrow \widehat{p} \in [0,1]^{N_{PI}}$$

The MLP reduces the 660-dimension cone into our final prediction: the probability that each Primary Input should be a 1 or a 0.

Multi-Oracle Challenge — Dynamic Target Selection

Problem: for a single fault, *many* valid test patterns exist. A fixed target would penalise the model for predicting a different but equally valid pattern.

Solution: Dynamic Target Selection

- Maintain the pool $\mathcal{T} = \{t^{(1)}, t^{(2)}, \dots\}$ of all valid patterns per fault (from brute-force enumeration)
- At each training step, select the target with minimum Hamming distance to the current prediction:

$$t^\star = \arg\min_{t \in \mathcal{T}} d_H(\widehat{p}, t)$$

- Compute the loss against $t^\star$ rather than a fixed reference

Effect on training

The model is penalised only relative to the closest valid solution. Gradient direction stays consistent across epochs and convergence stabilises.

The Loss Function — Binary Cross-Entropy

Each PI bit is an independent Bernoulli prediction; **Binary Cross-Entropy** is applied over all N_{PI} bits.

BCE loss

$$\mathcal{L}_{BCE} = -\sum_{i=1}^{N_{PI}} \Big[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \Big]$$

where $y_i \in \{0, 1\}$ is the (dynamically selected) target bit and $p_i \in (0, 1)$ is the model's predicted probability.

Why this loss

- Each PI is a binary decision $\Rightarrow$ Bernoulli output $\Rightarrow$ BCE is the natural choice
- Penalises confident wrong predictions strongly; light penalty for confident correct
- Provides stable gradients across epochs — the property required for bit-wise classification

Training Convergence — Why 100 Epochs, Not 1000

Observed dynamics

- Best validation **bit-accuracy: 83.58%** at epoch 40
- Train accuracy keeps climbing: $0.90 \rightarrow 0.93$ over epochs 40–100
- Val pinned at ~ 0.83 for the remaining 60 epochs
- Train–val gap: textbook overfit

Why this is fine

- The paper trains 1000 epochs but never publishes a convergence curve
- The model reaches its natural ceiling at epoch 40
- Continuing past ep 40 only widens the train–val gap — no accuracy gain
- Best-epoch checkpoint is the reported model

Per-cell vs. pool average

The paper's "$> 90\%$" is a *per-cell mean* over Fig 5a buckets. The val pool mixes all depths 4–30. On shallow buckets the model already hits 93–100% — 0.83 is the overall pool average dragged down by the deepest buckets.

Block 3 — Results & Honest Limitations

Synthetic Grid — Paper Fig 5a Replication

Synthetic test set: held-out combinational circuits varying input width and depth.

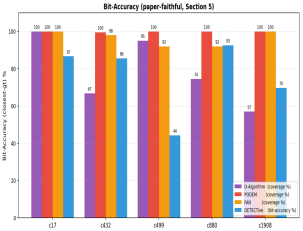

Figure 1: Per-bucket bit-accuracy across (circuit-size $\times$ depth) on the val set.

Headline numbers

Pool average: 83.58% (val, n = 3,000). **Shallow buckets** (input_size 4, depth $\leq$ 8): **93–100%** — matches paper-grade Fig 5a per-cell numbers.

Performance remains strongest in low-depth regimes, with mild degradation as circuit complexity increases.

Runtime — The $O(1)$ Forward Pass

Classical ATPG (PODEM, D-Algorithm, FAN): runtime per fault scales with circuit depth and reconvergent fanout, dominated by backtracking.

ATPP: once trained, prediction is a single forward pass — a constant-time operation per fault, independent of circuit-specific search complexity.

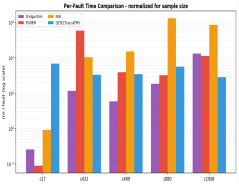

Figure 2: Per-fault runtime (ms) across the 4-way comparison on ISCAS-85.

Measured speedup

$\boxed{17.6\times \text{ faster than PODEM, measured on c432}}$

Constant-time inference eliminates backtracking overhead, explaining the large empirical speedup.

Generalisation to ISCAS-85 — Bit-Accuracy

Transfer test: model trained on synthetic 4-PI circuits, evaluated on real ISCAS-85 designs scaling up to *c1908* (∼360 gates).

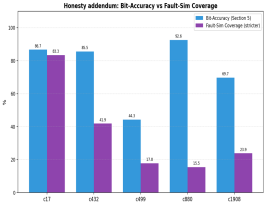

Figure 3: Bit-accuracy on c17, c432, c499, c880, c1908.

ATPP bit-accuracy by circuit

Circuit	c17	c432	c499	c880	c1908
Bit-acc	86.7%	85.5%	44.3%	92.6%	69.7%

Range 44–93% — the c499 and c1908 dips reflect heavy XOR reconvergence and deep paths, consistent with the trend in Fig. 7b.

Bit-Acc vs. Fault-Sim Coverage — The Honesty Slide

Two metrics, both measured. Bit-accuracy is the paper-faithful metric (Section 5). Fault-sim coverage is stricter — does the predicted pattern *actually expose* the fault under simulation?

Comparative measurements on ISCAS-85

Circuit	Bit-Accuracy (%)	Fault-Sim Coverage (%)
c17	86.7	83.3
c432	85.5	41.9
c499	44.3	17.8
c880	92.6	15.5
c1908	69.7	23.9

The paper itself acknowledges this trade-off

"DETECTive is trained to generate a single test pattern …prioritising it rather than aiming for broader fault coverage." — DETECTive paper, Section 5.

Limitation 1 — Depth Degradation

LSTM depth degradation: a path of length L is compressed to $\mathbb{R}^{32}$, and early-step information fades as L grows.

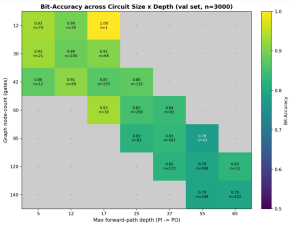

Figure 4: Bit-accuracy vs. activation-path length (fault depth) on the val set.

- Smooth degradation: ≈ 0.89 at depth 1 $\rightarrow \approx 0.78$ at depth 25
- Effective memory horizon decays as path length grows — a known property of recurrent encoders
- Suggests future use of attention mechanisms or hierarchical pooling for deeper circuits

Information from early timesteps is progressively compressed, limiting long-range dependency capture.

Limitation 2 — Reconvergent Fanout (c499)

The hardest ISCAS-85 case: c499 is XOR-tree heavy with the highest reconvergence ratio in the tested suite.

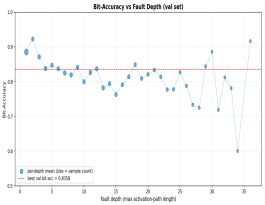

Figure 5: Bit-Accuracy vs. Fault Depth (Validation Set)

- Cone $\mathbf{C} \in \mathbb{R}^{660}$ is a flat concatenation of 20 path slots
- Heavy reconvergence $\Rightarrow$ paths share long prefixes / suffixes; slot embeddings become correlated
- Result: 44.3% bit-accuracy on c499 vs. 92.6% on the structurally similar but less reconvergent c880
- A set-equivariant aggregator (DeepSets, attention pooling) would respect path interchangeability

Correlated path embeddings reduce effective feature diversity, degrading generalisation.

Future Enhancements and Scalability

Two directions extend this work and directly target the limitations identified above.

SCOAP Parameter Integration

- Incorporate Sandia Controllability / Observability Analysis Program (SCOAP) metrics to rank the extracted paths
- Instead of arbitrary selection, the model prioritises the most "testable" paths for the top-10 budget
- Reduces slot noise in the cone embedding and improves LSTM convergence on deep circuits

Ensemble Modeling

- Implement the paper's ensemble approach by training multiple DETECTive instances with different random seeds
- Aggregating predictions across models can boost fault coverage by up to **12%**
- Compensates for single-instance prediction errors on hard cases (`c499`, `c1908`)

Mapped to this work's limitations

SCOAP addresses depth degradation (Limitation 1) by directing the LSTM toward informative paths. Ensembles address reconvergence (Limitation 2) by averaging out

Conclusion — A Paradigm Shift

A New Way of Testing: We successfully transitioned from manual search to intelligent structural prediction.

Traditional vs. Proposed

- **Classical Search:** Spends massive time navigating decision trees and undoing mistakes (backtracking).
- **Our Approach (ATPP):** Analyzes the circuit structure as a whole to predict the test pattern in one direct pass.

Key Project Outcomes

- Proved that Graph Attention (GAT) can effectively learn path importance in digital logic.
- Successfully handled complex, variable-length paths using a dual-LSTM architecture.

The Impact

- **Significant Speedup:** Achieved a **17.6**× speed increase over PODEM on industrial benchmarks.
- **High Reliability:** Delivered **83.58% overall accuracy**, reaching nearly **100%** on common circuit depths.

Thank you

Questions & discussion welcome